

05 — Batch Processing in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Batch INSERT Strategies](#)
 3. [COPY Performance](#)
 4. [Chunked Updates](#)
 5. [Upserts with ON CONFLICT](#)
 6. [Chunked Deletes](#)
 7. [Batch Processing Patterns](#)
 8. [Parallel Batch Processing](#)
 9. [Common Mistakes](#)
 10. [Best Practices](#)
 11. [Performance Considerations](#)
 12. [Interview Questions & Answers](#)
 13. [Exercises & Solutions](#)
 14. [Real-World Scenarios](#)
 15. [Cross-References](#)
-

Learning Objectives

- Choose the right batch INSERT strategy for different volume sizes
 - Perform chunked updates and deletes to avoid table bloat and lock contention
 - Implement efficient upserts using `ON CONFLICT`
 - Tune PostgreSQL parameters for batch workloads
 - Avoid long-running transactions in batch pipelines
-

Batch INSERT Strategies

Strategy Comparison

Single-row INSERT (worst)

for row in data:

 INSERT INTO t VALUES (row) ← 1 round-trip each, WAL flush each

Multi-row INSERT (good)

INSERT INTO t VALUES (r1),(r2),...,(r1000) ← 1 round-trip per 1000 rows

COPY (best for large volumes)

COPY t FROM STDIN ← stream, minimal WAL, no per-row overhead

Prepared statement batch (good for app-side)

PREPARE ins AS INSERT INTO t VALUES (\$1,\$2,\$3);

EXECUTE ins (v1, v2, v3); ← skips parse on repeat calls

Multi-row INSERT in Python (psycopg2)

```
import psycopg2
import psycopg2.extras

conn = psycopg2.connect(dsn="host=localhost dbname=mydb user=app")

# Method 1: execute_values (fastest Python bulk insert)
data = [(1, 'Alice', 100.0), (2, 'Bob', 200.0)] # ... thousands of tuples
with conn.cursor() as cur:
    psycopg2.extras.execute_values(
        cur,
        "INSERT INTO orders (id, name, amount) VALUES %s ON CONFLICT DO NOTHING",
        data,
        template="(%s, %s, %s)",
        page_size=1000 # batch size per INSERT statement
    )
conn.commit()

# Method 2: copy_from (fastest overall)
import io
buffer = io.StringIO()
for row in data:
    buffer.write('\t'.join(str(v) for v in row) + '\n')
buffer.seek(0)
with conn.cursor() as cur:
    cur.copy_from(buffer, 'orders', columns=('id', 'name', 'amount'))
conn.commit()
```

Multi-row INSERT in Java (JDBC batch)

```
String sql = "INSERT INTO orders (id, name, amount) VALUES (?, ?, ?)";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    conn.setAutoCommit(false);
    for (Order order : orders) {
        ps.setLong(1, order.getId());
        ps.setString(2, order.getName());
        ps.setBigDecimal(3, order.getAmount());
        ps.addBatch();

        if (ps.getBatchCount() % 1000 == 0) {
            ps.executeBatch();
            conn.commit();
        }
    }
    ps.executeBatch();
    conn.commit();
}
```

Bulk INSERT with unnest (pure SQL)

```
-- Insert arrays as rows – efficient from application code
INSERT INTO orders (id, customer_id, amount)
SELECT *
FROM UNNEST(
    ARRAY[1, 2, 3, 4, 5]::BIGINT[],
    ARRAY[101, 102, 103, 101, 105]::INT[],
    ARRAY[10.0, 20.5, 15.0, 30.0, 8.75]::NUMERIC[]
);
```

COPY Performance

COPY Benchmarking

```
-- Measure COPY throughput
\timing on

COPY large_table FROM '/tmp/large_data.csv' WITH (FORMAT CSV, HEADER);

-- Expected: 300K-500K rows/second on modern hardware
```

COPY with pre-processing

```
-- Disable FK checks during COPY (re-enable after)
SET session_replication_role = replica; -- disables trigger-based FK enforcement
COPY orders FROM '/tmp/orders.csv' WITH (FORMAT CSV, HEADER);
SET session_replication_role = DEFAULT;
ANALYZE orders;

-- Disable autovacuum on target table during heavy load
ALTER TABLE orders SET (autovacuum_enabled = false);
COPY orders FROM '/tmp/orders.csv' WITH (FORMAT CSV, HEADER);
ALTER TABLE orders SET (autovacuum_enabled = true);
VACUUM ANALYZE orders;
```

Streaming COPY from Python (memory efficient)

```
import psycopg2
import csv

def stream_copy_to_postgres(conn, table, filename, columns):
    """Stream large CSV to PostgreSQL using COPY without loading into memory."""
    with open(filename, 'r') as f:
        reader = csv.reader(f)
        next(reader) # skip header
```

```

with conn.cursor() as cur:
    cur.copy_expert(
        f"COPY {table} ({','.join(columns)}) FROM STDIN WITH CSV",
        f
    )
conn.commit()

```

Chunked Updates

Large UPDATES lock rows and generate massive WAL. Break them into chunks:

```

-- Anti-pattern: update 50M rows in one transaction
UPDATE orders SET status = 'archived' WHERE created_at < '2020-01-01';
-- ↑ Holds locks for hours, generates huge WAL, autovacuum can't keep up

-- Better: chunked update with key-based cursor
DO $$
DECLARE
    v_batch_size INT := 10000;
    v_last_id BIGINT := 0;
    v_max_id BIGINT;
    v_rows INT;
BEGIN
    SELECT MAX(order_id) INTO v_max_id FROM orders WHERE created_at < '2020-01-01';

    LOOP
        UPDATE orders
        SET status = 'archived'
        WHERE order_id > v_last_id
            AND order_id <= v_last_id + v_batch_size
            AND created_at < '2020-01-01'
            AND status != 'archived'; -- skip already done

        GET DIAGNOSTICS v_rows = ROW_COUNT;
        EXIT WHEN v_rows = 0 OR v_last_id > v_max_id;

        v_last_id := v_last_id + v_batch_size;

        RAISE NOTICE 'Updated through id %, % rows this batch', v_last_id, v_rows;
        PERFORM pg_sleep(0.1); -- brief pause to let autovacuum breathe
    END LOOP;

    RAISE NOTICE 'Chunked update complete';
END;
$$;

```

Chunked UPDATE in Python

```

import psycopg2, time

def chunked_update(conn, batch_size=10000, sleep_ms=100):
    last_id = 0
    total = 0
    while True:
        with conn.cursor() as cur:
            cur.execute("""
                UPDATE orders
                SET status = 'archived'
                WHERE order_id IN (
                    SELECT order_id FROM orders
                    WHERE order_id > %s
                    AND created_at < '2020-01-01'
                    AND status != 'archived'
                    ORDER BY order_id
                    LIMIT %s
                )
                RETURNING order_id
            """, (last_id, batch_size))
            updated = cur.fetchall()
            conn.commit()

            if not updated:
                break
            last_id = updated[-1][0]
            total += len(updated)
            print(f"Updated {total} rows, last_id={last_id}")
            time.sleep(sleep_ms / 1000)

    print(f"Complete: {total} rows updated")

```

Upserts with ON CONFLICT

```

-- Basic upsert: insert or update on PK conflict
INSERT INTO products (product_id, name, price, stock)
VALUES (101, 'Widget', 9.99, 500)
ON CONFLICT (product_id) DO UPDATE
SET
    name = EXCLUDED.name,
    price = EXCLUDED.price,
    stock = EXCLUDED.stock,
    updated_at = NOW();

-- Conditional upsert: only update if price changed
INSERT INTO products (product_id, name, price, stock)
VALUES (101, 'Widget', 9.99, 500)
ON CONFLICT (product_id) DO UPDATE
SET price = EXCLUDED.price,

```

```

        updated_at = NOW()
WHERE products.price <> EXCLUDED.price;    -- skip unchanged rows

-- Upsert with partial conflict (unique index)
CREATE UNIQUE INDEX idx_order_product ON order_items (order_id, product_id);

INSERT INTO order_items (order_id, product_id, quantity)
VALUES (1001, 55, 3)
ON CONFLICT (order_id, product_id) DO UPDATE
SET quantity = order_items.quantity + EXCLUDED.quantity;    -- increment quantity

-- Upsert ignoring conflicts
INSERT INTO events (event_id, user_id, type)
VALUES (gen_random_uuid(), 42, 'page_view')
ON CONFLICT DO NOTHING;    -- ignore all conflicts

```

Batch upsert with unnest

```

-- High-performance batch upsert from arrays
INSERT INTO products (product_id, name, price)
SELECT *
FROM UNNEST(
    ARRAY[101, 102, 103]::INT[],
    ARRAY['Widget', 'Gadget', 'Doohickey']::TEXT[],
    ARRAY[9.99, 19.99, 4.99]::NUMERIC[]
)
ON CONFLICT (product_id) DO UPDATE
SET name = EXCLUDED.name,
    price = EXCLUDED.price;

```

Chunked Deletes

```

-- Chunked delete (avoid long locks and massive WAL)
DO $$
DECLARE
    v_deleted INT := 1;
BEGIN
    WHILE v_deleted > 0 LOOP
        DELETE FROM events
        WHERE event_id IN (
            SELECT event_id FROM events
            WHERE created_at < NOW() - INTERVAL '90 days'
            LIMIT 10000
        );
        GET DIAGNOSTICS v_deleted = ROW_COUNT;
        PERFORM pg_sleep(0.05);
    END LOOP;
END;
$$;

```

```
-- Alternative: partition drop (fastest delete)
-- If events is partitioned by month:
ALTER TABLE events DETACH PARTITION events_2023_09;
DROP TABLE events_2023_09; -- instant, no WAL for row deletions
```

Batch Processing Patterns

Batch job table pattern

```
CREATE TABLE batch_jobs (
  job_id      BIGSERIAL PRIMARY KEY,
  job_type    TEXT      NOT NULL,
  status      TEXT      NOT NULL DEFAULT 'pending', -- pending, running,
done, failed
  batch_date  DATE,
  parameters  JSONB,
  started_at  TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  rows_processed INT,
  error_detail TEXT
);

-- Claim next job (safe concurrent batch workers)
WITH claimed AS (
  SELECT job_id FROM batch_jobs
  WHERE status = 'pending'
  ORDER BY job_id
  LIMIT 1
  FOR UPDATE SKIP LOCKED -- skip jobs locked by other workers
)
UPDATE batch_jobs
SET status = 'running', started_at = NOW()
FROM claimed
WHERE batch_jobs.job_id = claimed.job_id
RETURNING batch_jobs.*;
```

Processing queue with SKIP LOCKED

```
-- Multiple workers process from a work queue concurrently
CREATE TABLE work_queue (
  id          BIGSERIAL PRIMARY KEY,
  payload     JSONB      NOT NULL,
  status      TEXT      NOT NULL DEFAULT 'pending',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Worker picks up next item (safe for concurrent workers)
BEGIN;
```

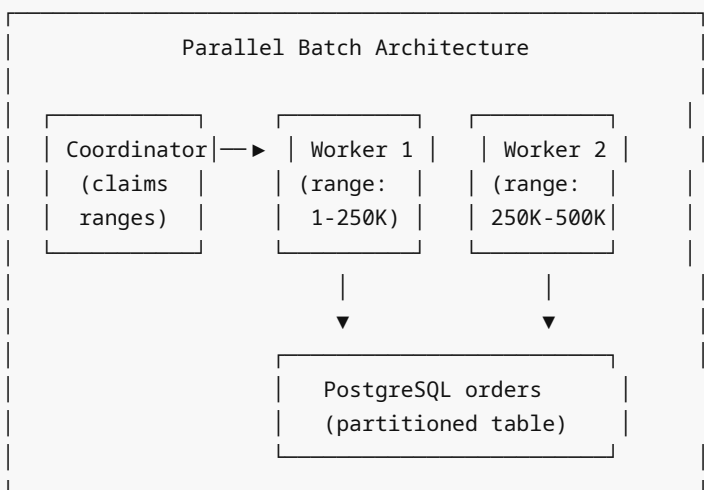
```

SELECT id, payload
FROM work_queue
WHERE status = 'pending'
ORDER BY created_at
LIMIT 1
FOR UPDATE SKIP LOCKED;

-- Process payload, then:
UPDATE work_queue SET status = 'done' WHERE id = :claimed_id;
COMMIT;

```

Parallel Batch Processing



```

#!/bin/bash
# Parallel batch update using range partitioning
WORKERS=8
TOTAL_ROWS=8000000
CHUNK=$((TOTAL_ROWS / WORKERS))

for i in $(seq 0 $((WORKERS-1))); do
    FROM_ID=$((i * CHUNK + 1))
    TO_ID=$((i + 1) * CHUNK)

    psql -d mydb -c "
        UPDATE orders
        SET status = 'processed'
        WHERE order_id BETWEEN $FROM_ID AND $TO_ID
        AND status = 'pending'
    " &
done
wait
echo "Parallel batch complete"

```


Common Mistakes

Mistake	Impact	Fix
Single huge UPDATE transaction	Long lock time, table bloat	Chunked updates with COMMIT per batch
Not using SKIP LOCKED in queue	Workers deadlock	FOR UPDATE SKIP LOCKED
Batch size too large	Memory pressure, slow commits	1000-10000 rows per batch
Batch size too small	Too many round-trips	Use execute_values with page_size=1000
Upsert on non-indexed conflict column	Full table scan per conflict	Index the conflict column
Not running VACUUM after large batch deletes	Table bloat, slow subsequent queries	VACUUM ANALYZE after large deletes
synchronous_commit=off without idempotency	Data loss on crash	Only for idempotent, re-playable loads

Best Practices

1. **Chunk size sweet spot:** 1,000-10,000 rows per transaction.
2. **Add sleep between chunks** (50-100ms) to allow autovacuum to keep pace.
3. **Use SKIP LOCKED** for any queue-based batch processing.
4. **Prefer partition DROP** over DELETE for time-based purges.
5. **Monitor autovacuum:** pause batch jobs if autovacuum is severely lagging.
6. **Track batch progress** in a batch_jobs table for observability.
7. **COPY >> multi-row INSERT >> single-row INSERT** in performance order.

Performance Considerations

```
-- Tune for batch workloads (session-level)
SET work_mem = '256MB';
SET maintenance_work_mem = '1GB';
SET synchronous_commit = off;           -- only for idempotent loads

-- Monitor batch job performance
SELECT
    pid,
    query_start,
    now() - query_start AS duration,
    LEFT(query, 80) AS query_snippet,
    state
FROM pg_stat_activity
WHERE state != 'idle'
```

```
ORDER BY duration DESC;

-- Check table dead tuple ratio (bloat from batch updates)
SELECT
    relname,
    n_dead_tup,
    n_live_tup,
    ROUND(n_dead_tup::NUMERIC / NULLIF(n_live_tup + n_dead_tup, 0) * 100, 2) AS
dead_pct,
    last_autovacuum
FROM pg_stat_user_tables
WHERE relname IN ('orders', 'events')
ORDER BY dead_pct DESC;
```

Interview Questions & Answers

Q1: Why is single-row INSERT in a loop so slow?

Each INSERT involves a network round-trip, query parse, plan, execute, and WAL flush. At 1ms per round-trip, 1M rows takes 1000 seconds. Multi-row INSERT or COPY reduces this to seconds.

Q2: When would you use ON CONFLICT DO NOTHING vs DO UPDATE ?

DO NOTHING silently ignores conflicts — useful for idempotent event logs where re-inserting is safe. DO UPDATE merges the new data into the existing row — useful for upserts where you want the latest version.

Q3: How do chunked updates reduce table bloat?

Each committed batch allows autovacuum to reclaim dead tuples between chunks. A single massive UPDATE generates millions of dead tuples that autovacuum can't clean until the transaction commits.

Q4: What is FOR UPDATE SKIP LOCKED and why is it important for batch workers?

SKIP LOCKED skips any rows currently locked by other transactions, preventing deadlocks in concurrent queue processors. Without it, Worker A locks row 1; Worker B tries to lock row 1 and waits (or deadlocks).

Q5: What is the difference between execute_values and executemany in psycopg2?

executemany runs one INSERT per row. execute_values batches rows into a single multi-row INSERT statement, reducing round-trips by ~1000x.

Q6: How do you safely delete 100M rows from a table in production?

Chunked delete in loops of 10K rows with COMMIT and brief sleep between chunks. If the table is partitioned by date, simply DROP or DETACH the old partition — instant and zero WAL.

Q7: What is the UNNEST trick for batch upserts?

Arrays of values are passed from the application as PostgreSQL arrays, then UNNEST() turns them into rows. A single INSERT ... SELECT FROM UNNEST(...) ON CONFLICT DO UPDATE statement handles thousands of rows in one round-trip.

Q8: What happens to autovacuum during a long-running batch job?

Autovacuum is blocked from removing dead tuples created by the batch until the long transaction commits. Meanwhile, dead tuple ratio grows, table bloat accumulates, and subsequent queries slow down.

Exercises & Solutions

Exercise 1: Chunked archive

Task: Archive orders older than 1 year to `orders_archive` in chunks.

```
DO $$
DECLARE v_moved INT := 1;
BEGIN
    WHILE v_moved > 0 LOOP
        WITH moved AS (
            DELETE FROM orders
            WHERE order_id IN (
                SELECT order_id FROM orders
                WHERE created_at < NOW() - INTERVAL '1 year'
                LIMIT 5000
            )
            RETURNING *
        )
        INSERT INTO orders_archive SELECT * FROM moved;
        GET DIAGNOSTICS v_moved = ROW_COUNT;
        PERFORM pg_sleep(0.1);
    END LOOP;
END;
$$;
```

Exercise 2: Batch upsert with UNNEST

```
-- Upsert 5 products in one statement
INSERT INTO products (product_id, name, price)
SELECT id, nm, pr
FROM UNNEST(
    ARRAY[1,2,3,4,5]::INT[],
    ARRAY['A','B','C','D','E']::TEXT[],
    ARRAY[1.0,2.0,3.0,4.0,5.0]::NUMERIC[]
) AS t(id, nm, pr)
ON CONFLICT (product_id) DO UPDATE
    SET name=EXCLUDED.name, price=EXCLUDED.price;
```

Real-World Scenarios

Scenario: Nightly status recalculation 500K orders need status recalculated each night. Chunked UPDATE with `batch_size=10000` and 50ms sleep runs in ~45 minutes without impacting OLTP response times.

Scenario: Event processing queue A microservice drops events into `work_queue`. 16 worker threads run concurrently, each claiming 1 event at a time with `FOR UPDATE SKIP LOCKED`. Throughput: 8,000

events/second.

Cross-References

- `01_etl_with_postgresql.md` — COPY for bulk loads
- `06_incremental_loading.md` — watermark-based incremental batches
- `../17_PostgreSQL_For_Backend_Engineers/05_api_database_patterns.md` — pagination in batch contexts